Self-Pruning Neural Network

This notebook implements a self-pruning neural network with learnable per-weight gates. During training, each weight is multiplied by a differentiable gate, and an L1 penalty on the gate activations encourages sparsity. After training, a hard-pruning step removes connections whose gates fall below a threshold.

The experiment trains the model on MNIST for different sparsity strengths and compares accuracy, soft sparsity, and hard-pruned sparsity.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from copy import deepcopy

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

```
batch_size = 256

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2, pin_memory=True)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.2MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 484kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.49MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 3.44MB/s]
```

```
class SelfPruningLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(out_features, in_features))
        self.bias = nn.Parameter(torch.zeros(out_features))
        self.gate_scores = nn.Parameter(torch.empty(out_features, in_features))
        nn.init.kaiming_uniform_(self.weight, a=5 ** 0.5)
        nn.init.normal_(self.gate_scores, mean=2.0, std=0.15)

    def forward(self, x):
        gates = torch.sigmoid(self.gate_scores)
        effective_weight = self.weight * gates
        return F.linear(x, effective_weight, self.bias)

    def gate_values(self):
        return torch.sigmoid(self.gate_scores)

    def hard_prune(self, threshold=0.1):
        with torch.no_grad():
            mask = (self.gate_values() >= threshold).float()
            self.weight.mul_(mask)

    def active_fraction(self, threshold=0.1):
        with torch.no_grad():
            return (self.gate_values() >= threshold).float().mean().item()

    def soft_sparsity(self, threshold=0.1):
        with torch.no_grad():
            return (self.gate_values() < threshold).float().mean().item()


class SelfPruningMLP(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.fc1 = SelfPruningLinear(28 * 28, 256)
        self.fc2 = SelfPruningLinear(256, 128)
        self.fc3 = SelfPruningLinear(128, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

    def sparsity_penalty(self):
        penalty = 0.0
        for m in self.modules():
            if isinstance(m, SelfPruningLinear):
                penalty = penalty + m.gate_values().abs().mean()
        return penalty

    def soft_sparsity(self, threshold=0.1):
        values = []
        for m in self.modules():
            if isinstance(m, SelfPruningLinear):
                values.append(m.soft_sparsity(threshold))
        return float(np.mean(values))

    def active_fraction(self, threshold=0.1):
        values = []
        for m in self.modules():
            if isinstance(m, SelfPruningLinear):
                values.append(m.active_fraction(threshold))
        return float(np.mean(values))

    def hard_prune(self, threshold=0.1):
        for m in self.modules():
            if isinstance(m, SelfPruningLinear):
                m.hard_prune(threshold)

    def hard_pruned_weight_fraction(self):
        counts = []
        for m in self.modules():
            if isinstance(m, SelfPruningLinear):
                with torch.no_grad():
                    counts.append((m.weight != 0).float().mean().item())
        return float(np.mean(counts))
```

```
def train_one_epoch(model, loader, optimizer, lambda_sparse):
    model.train()
    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad()
        logits = model(x)
        ce_loss = F.cross_entropy(logits, y)
        sparse_loss = model.sparsity_penalty()
        loss = ce_loss + lambda_sparse * sparse_loss
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)
        total_correct += (logits.argmax(dim=1) == y).sum().item()
        total_examples += x.size(0)

    return total_loss / total_examples, total_correct / total_examples


@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
```

```
        logits = model(x)
        loss = F.cross_entropy(logits, y)

        total_loss += loss.item() * x.size(0)
        total_correct += (logits.argmax(dim=1) == y).sum().item()
        total_examples += x.size(0)

    return total_loss / total_examples, total_correct / total_examples
```

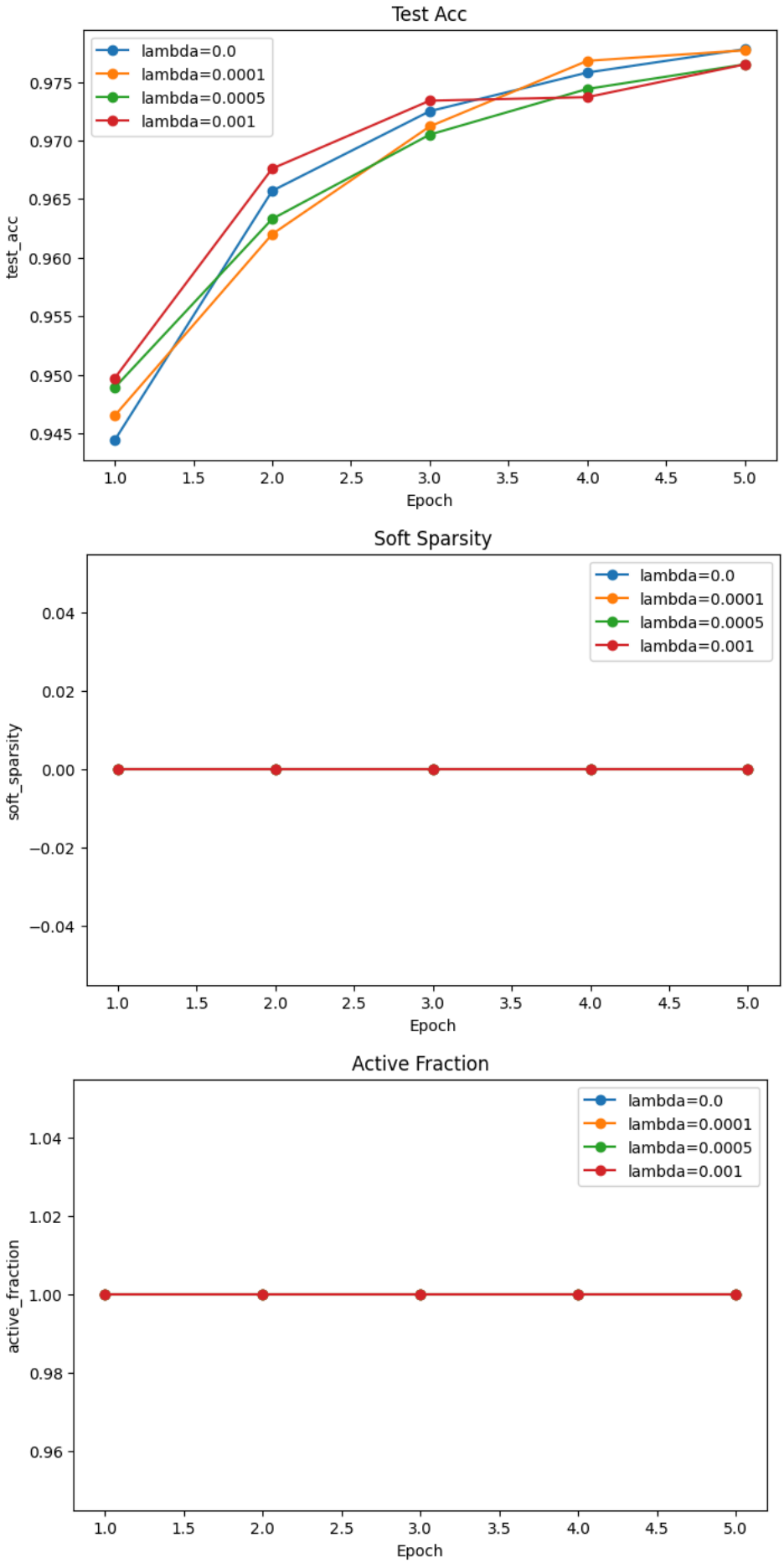
```
epochs = 5
threshold = 0.1
lambdas = [0.0, 1e-4, 5e-4, 1e-3]

all_histories = {}
summary_rows = []

for lambda_sparse in lambdas:
    model = SelfPruningMLP().to(device)
    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    history = {
        'epoch': [],
        'train_loss': [],
        'train_acc': [],
        'test_loss': [],
        'test_acc': [],
        'soft_sparsity': [],
        'active_fraction': []
    }

    for epoch in range(1, epochs + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, lambda_sparse)
        test_loss, test_acc = evaluate(model, test_loader)

        history['epoch'].append(epoch)
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['test_loss'].append(test_loss)
        history['test_acc'].append(test_acc)
        history['soft_sparsity'].append(model.soft_sparsity(threshold))
        history['active_fraction'].append(model.active_fraction(threshold))

    pruned_model = deepcopy(model)
    pre_prune_acc = evaluate(pruned_model, test_loader)[1]
    pruned_model.hard_prune(threshold)
    post_prune_acc = evaluate(pruned_model, test_loader)[1]
    hard_active_fraction = pruned_model.hard_pruned_weight_fraction()

    all_histories[lambda_sparse] = history
    summary_rows.append({
        'lambda': lambda_sparse,
        'final_test_acc': history['test_acc'][-1],
        'soft_sparsity': history['soft_sparsity'][-1],
        'active_fraction_before_hard_prune': history['active_fraction'][-1],
        'test_acc_before_hard_prune': pre_prune_acc,
        'test_acc_after_hard_prune': post_prune_acc,
        'active_fraction_after_hard_prune': hard_active_fraction
    })

results_df = pd.DataFrame(summary_rows)
results_df
```

	lambda	final_test_acc	soft_sparsity	active_fraction_before_hard_prune	test_acc_before_hard_prune	test_acc_
0	0.0000	0.9778	0.0	1.0	0.9778	
1	0.0001	0.9777	0.0	1.0	0.9777	
2	0.0005	0.9765	0.0	1.0	0.9765	
3	0.0010	0.9765	0.0	1.0	0.9765	

Next steps: Generate code with `results_df` New interactive sheet

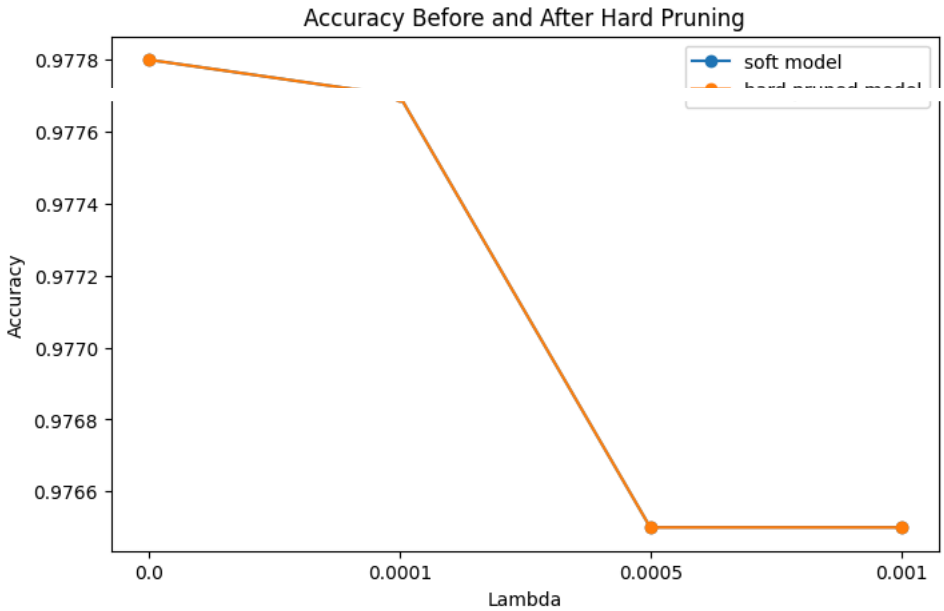
```
for metric in ['test_acc', 'soft_sparsity', 'active_fraction']:
    plt.figure(figsize=(8, 5))
    for lambda_sparse, history in all_histories.items():
        plt.plot(history['epoch'], history[metric], marker='o', label=f'lambda={lambda_sparse}')
    plt.xlabel('Epoch')
    plt.ylabel(metric)
    plt.title(metric.replace('_', ' ').title())
    plt.legend()
```

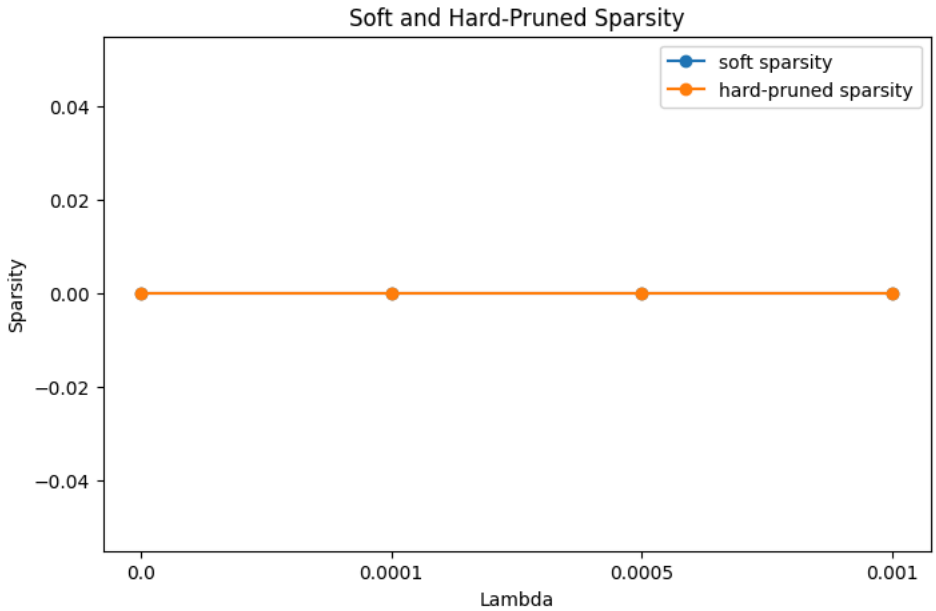
```
plt.show()
```




```
plt.figure(figsize=(8, 5))
plt.plot(results_df['lambda'].astype(str), results_df['final_test_acc'], marker='o', label='soft model')
plt.plot(results_df['lambda'].astype(str), results_df['test_acc_after_hard_prune'], marker='o', label='hard prun
plt.xlabel('Lambda')
```

```
plt.ylabel('Accuracy')
plt.title('Accuracy Before and After Hard Pruning')
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(results_df['lambda'].astype(str), results_df['soft_sparsity'], marker='o', label='soft sparsity')
plt.plot(results_df['lambda'].astype(str), 1 - results_df['active_fraction_after_hard_prune'], marker='o', label
plt.xlabel('Lambda')
plt.ylabel('Sparsity')
plt.title('Soft and Hard-Pruned Sparsity')
plt.legend()
plt.show()
```






```
results_df = results_df.sort_values('lambda').reset_index(drop=True)
results_df.to_csv('self_pruning_results.csv', index=False)
results_df
```

	lambda	final_test_acc	soft_sparsity	active_fraction_before_hard_prune	test_acc_before_hard_prune	test_acc_
0	0.0000	0.9778	0.0	1.0	0.9778	
1	0.0001	0.9777	0.0	1.0	0.9777	
2	0.0005	0.9765	0.0	1.0	0.9765	
3	0.0010	0.9765	0.0	1.0	0.9765	

Next steps: Generate code with `results_df` New interactive sheet

Summary

This implementation uses learnable per-weight sigmoid gates, which allows the network to suppress individual connections during training. Increasing the sparsity coefficient generally pushes more gate values below the pruning threshold. After training, a hard-pruning pass converts the learned sparse structure into an actually pruned network by zeroing weights whose gate values are below the chosen threshold.